

Reviewer Pack — Minimal Rank of Geometric Langlands Sheaves Bounds ACC⁰ Circ...

Entry 3e82705f1beb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 14:23:57 UTC

Minimal Rank of Geometric Langlands Sheaves Bounds ACC⁰ Circuit Weights

Entry ID: 3e82705f1beb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 14:23:57 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program (Sheaves) **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

{‘sentence_1’: ‘For a Boolean function f with n inputs, the minimal rank of the sheaf corresponding to f on an appropriate curve is upper bounded by a function of n .’, ‘sentence_2’: ‘This bound provides a direct link between geometric Langlands theory and ACC⁰ circuit weights.’, ‘sentence_3’: ‘The conjecture states that the minimal rank of the sheaf grows logarithmically with n , implying that the corresponding Boolean function can be computed by an ACC⁰ circuit.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Geometric Langlands theory provides a bridge between geometry and representation theory, potentially offering new insights into the complexity of computational tasks.’, ‘sentence_2’: ‘The connection with ACC⁰ circuits suggests that geometric Langlands sheaves could be used to analyze the structure of Boolean functions.’, ‘sentence_3’: ‘This conjecture aims to establish a quantitative relationship between these fields and could lead to new algorithms or theoretical results.’}

Taxonomy category: GeometricLanglandsACC0CircuitWeights (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 01c7f7a41f691418

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the minimal rank of the sheaf grows logarithmically with n (i.e., $\log(n) \leq \text{minimal_rank} \leq 2\log(n)$), and falsified if there exists a Boolean function f with n inputs where the minimal rank exceeds $2\log(n)$ or any seed produces a minimal rank less than $\log(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric Langlands program sheaves" AND "Boolean function complexity" - "minimal rank sheaf" AND "ACC⁰ circuit weights" - "logarithmic growth sheaf" AND Boolean function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20 # Fixed size for simplicity, can be adjusted if needed
    instances_tested = 30

    def construct_sheaf(f):
        sheaf = []
        for i in range(n):
            row = [f[j] if j & (1 << i) else 0 for j in range(2**n)]
            sheaf.append(row)
        return sheaf

    def compute_minimal_rank(sheaf):
        m, n = len(sheaf), len(sheaf[0])
        A = [[Fraction(sheaf[i][j]) for j in range(n)] for i in range(m)]

        def gaussian_elimination(A):
            rows, cols = len(A), len(A[0])
            rank = 0
            for col in range(cols):
                pivot_row = -1
                for row in range(rank, rows):
                    if A[row][col] != 0:
                        pivot_row = row
                        break
                if pivot_row == -1:
```

```

        continue
    A[pivot_row], A[rank] = A[rank], A[pivot_row]
    rank += 1
    for row in range(rank, rows):
        factor = A[row][col] / A[pivot_row][col]
        for j in range(col, cols):
            A[row][j] -= factor * A[pivot_row][j]
    return rank

return gaussian_elimination(A)

def generate_random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

total_rank = 0
counterexample = ""
conjecture_holds = True

for _ in range(instances_tested):
    f = generate_random_boolean_function(n)
    sheaf = construct_sheaf(f)
    rank = compute_minimal_rank(sheaf)

    if rank < math.log2(2**n) or rank > 2 * math.log2(2**n):
        conjecture_holds = False
        counterexample = f"Function with n={n} and rank {rank}"
        break

    total_rank += rank

return {
    "metric_name": "minimal_rank",
    "metric_value": total_rank / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_rank = math.sqrt(sum((result["metric_value"] - mean_rank)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

```
else:
    print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the minimal rank of the sheaf grows logarithmically with n as required by the conjecture. | next: Run the test again under controlled conditions to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16733
2	preregistration	ollama_remote	glm4:latest	0	0	5759
3	novelty	ollama_remote	glm4:latest	0	0	4601
4	novelty	ollama_remote	glm4:latest	0	0	5693
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14749
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10629
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11864
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9791
9	judge	ollama_remote	glm4:latest	0	0	9946

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89765 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3e82705f1beb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/3e82705f1beb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3e82705f1beb.tar.gz` (if generated)